

Отчет по лабораторной работе №13

Архитектура компьютера

Агапова Анна Антоновна

Содержание

1	Цель работы	5
2	Задание	6
3	Выполнение лабораторной работы	7
4	Выводы	16
	Ответы на контрольные вопросы	17

Список иллюстраций

3.1	Создание файла	7
3.2	Код	8
3.3	Создание файла.....	8
3.4	Результат	9
3.5	Создание файла.....	9
3.6	Код	10
3.7	Код	11
3.8	Результат	11
3.9	Создание файла.....	12
3.10	Код.....	12
3.11	Результат	13
3.12	Создание файла.....	14
3.13	Код.....	14
3.14	Проверка.....	15
3.15	Результат	15

Список таблиц

1 Цель работы

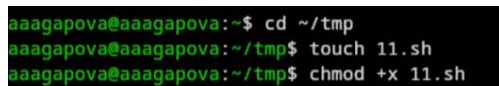
Изучить основы программирования в оболочке ОС UNIX. Научится писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

2 Задание

1. Используя команды `getopts` `grep`, написать командный файл, который анализирует командную строку с ключами: `-i` — прочитать данные из указанного файла; `-o` — вывести данные в указанный файл; `-r` — указать шаблон для поиска; `-C` — различать большие и малые буквы; `-n` — выдавать номера строк. а затем ищет в указанном файле нужные строки, определяемые ключом `-r`.
2. Написать на языке Си программу, которая вводит число и определяет, является ли оно больше нуля, меньше нуля или равно нулю. Затем программа завершается с помощью функции `exit(n)`, передавая информацию в о коде завершения в оболочку. Командный файл должен вызывать эту программу и, проанализировав с помощью команды `S?`, выдать сообщение о том, какое число было введено.
3. Написать командный файл, создающий указанное число файлов, пронумерованных последовательно от 1 до N (например `1.tmp`, `2.tmp`, `3.tmp`, `4.tmp` и т.д.). Число файлов, которые необходимо создать, передаётся в аргументы командной строки. Этот же командный файл должен уметь удалять все созданные им файлы (если они существуют).
4. Написать командный файл, который с помощью команды `tag` запаковывает в архив все файлы в указанной директории. Модифицировать его так, чтобы запаковывались только те файлы, которые были изменены менее недели тому назад (использовать команду `find`).

3 Выполнение лабораторной работы

1. Создаю исполняемый файл для первого задания. (рис. 3.1).

A terminal window with a black background and green text. It shows three lines of commands being executed in a shell. The first line is 'cd ~/tmp', the second is 'touch 11.sh', and the third is 'chmod +x 11.sh'. Each line is preceded by the prompt 'aaagarova@aaagarova:~\$' or 'aaagarova@aaagarova:~/tmp\$'.

```
aaagarova@aaagarova:~$ cd ~/tmp
aaagarova@aaagarova:~/tmp$ touch 11.sh
aaagarova@aaagarova:~/tmp$ chmod +x 11.sh
```

Рис. 3.1: Создание файла

2. Пишу программу. (рис. 3.2).

```
#!/bin/bash

cflag=
nflag=
pval=
ival=
oval=

while getopts i:o:p:Cn optletter
do
    case $optletter in
        C) cflag=-i;;
        n) nflag=-n;;
        p) pval="-e $OPTARG";;
        i) ival="$OPTARG";;
        o) oval="$OPTARG";;
        *) echo "Illegal option $optletter";;
    esac
done

if test -n "$cflag"
then
    cf=-i
fi

if test -n "$nflag"
then
    nf=-n
fi

grep $cf $nf $pval $ival >> $oval
```

Рис. 3.2: Код

3. Создаю файл input.txt. (рис. 3.3).

```
Изучить основы программирования в оболочке ОС UNIX. hi|
Научится hi писать более
сложные командные файлы с использованием логических
управляющих конструкций
и циклов. hi
```

Рис. 3.3: Создание файла

4. Результат работы программы. (рис. 3.4).

```
aaagapova@aaagapova:~/tmp$ ./11.sh -i input.txt -p hi -o output.txt
aaagapova@aaagapova:~/tmp$ cat output.txt
Изучить основы программирования в оболочке ОС UNIX. hi
Научится hi писать более
и циклов. hi
```

Рис. 3.4: Результат

5. Создаю исполняемый файл для второго задания. (рис. 3.5).

```
aaagapova@aaagapova:~/tmp$ touch 12.sh
aaagapova@aaagapova:~/tmp$ chmod +x 12.sh
aaagapova@aaagapova:~/tmp$ touch 12.cpp
aaagapova@aaagapova:~/tmp$ bash 12.sh
```

Рис. 3.5: Создание файла

6. Пишу программу в файле 12.c. (рис. 3.6).

```
#include <stdlib.h>
#include <stdio.h>

int main() {
    int n;
    printf("Введите число: ");
    scanf("%d", &n);
    if (n > 0) {
        exit(1);
    }
    else if (n == 0) {
        exit(0);
    }
    else {
        exit(2);
    }
}
```

Рис. 3.6: Код

7. Пишу программу в файле 12.sh. (рис. 3.7).

```
#!/bin/bash
gcc -o cprog 12.c
./cprog
case $? in
0) echo "Число равно нулю" ;;
1) echo "Число больше нуля" ;;
2) echo "Число меньше нуля" ;;
esac
```

Рис. 3.7: Код

8. Проверяю работу программы двумя способами. (рис. 3.8).

```
aaagarova@aaagarova:~/tmp$ bash 12.sh
Введите число: 34
Число больше нуля
aaagarova@aaagarova:~/tmp$ ./12.sh
Введите число: 76
Число больше нуля
```

Рис. 3.8: Результат

9. Создаю исполняемый файл для третьего задания. (рис. 3.9).

```
aaagapova@aaagapova:~/tmp$ touch 13.sh  
aaagapova@aaagapova:~/tmp$ chmod +x 13.sh
```

Рис. 3.9: Создание файла

10. Пишу программу. (рис. 3.10).

```
#!/bin/bash  
  
for (( i=1; i<=$*; i++ ))  
do  
    if test -f "$i".tmp  
    then  
        rm "$i".tmp  
    else  
        touch "$i.tmp"  
    fi  
done
```

Рис. 3.10: Код

11. Проверяю работу программы. (рис. 3.11).

```

aaagapova@aaagapova:~/tmp$ bash 13.sh 4
aaagapova@aaagapova:~/tmp$ ls
004-lab_shell.pdf  12.c  13.sh  2.tmp  4.tmp  input.txt  output.txt  shakespeare.tar.gz
11.sh             12.sh  1.tmp  3.tmp  cprog  love_lines.txt  Shakespeare
aaagapova@aaagapova:~/tmp$ bash 13.sh 4
aaagapova@aaagapova:~/tmp$ ls
004-lab_shell.pdf  12.c  13.sh  input.txt  output.txt  shakespeare.tar.gz
11.sh             12.sh  cprog  love_lines.txt  Shakespeare

```

Рис. 3.11: Результат

12. Создаю исполняемый файл для четвертого задания. (рис. 3.12).

```
aaagapova@aaagapova:~/tmp$ touch 14.sh  
aaagapova@aaagapova:~/tmp$ chmod +x 14.sh
```

Рис. 3.12: Результат

13. Пишу программу. (рис. 3.13).

```
#!/bin/bash  
  
find $* -mtime -7 -mtime +0 -type f > FILES.txt  
tar -cf archive.tar -T FILES.txt
```

Рис. 3.13: Код

14. Проверяю работу программы. (рис. 3.14).

```
aaagapova@aaagapova:~/tmp$ bash 14.sh /home/aaagapova/tmp
```

Рис. 3.14: Проверка

15. Архив создался. (рис. 3.15).



Рис. 3.15: Результат

4 Выводы

Я изучила основы программирования в оболочке ОС UNIX, научилась писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

Ответы на контрольные вопросы

1. `getopts` — команда для обработки флагов (опций) командной строки. Она считывает ключи вроде `-i file.txt` или `-n` и присваивает их значения переменным. Обычно используется в цикле `while` вместе с `case`.
2. Метасимволы — это спецсимволы для поиска файлов по маске:
 - - — заменяет любую строку (хоть пустую)
 - `?` — заменяет ровно один символ
 - `[a-z]` — один символ из диапазона
3. Основные конструкции `bash`:
 - `if then else fi` — условие (если-то-иначе)
 - `case in esac` — выбор из нескольких вариантов
 - `for do done` — цикл по списку
 - `while do done` — цикл пока условие истинно
 - `until do done` — цикл пока условие ложно
4. `break` — полностью выходит из цикла, `continue` — пропускает оставшуюся часть текущей итерации и переходит к следующей
5. `true` — всегда возвращает успех (0), означает “истина”, `false` — всегда возвращает ошибку (не 0), означает “ложь”. Нужны для условных конструкций. Например, `while true; do ... done` — бесконечный цикл.
6. Проверяет, существует ли обычный файл по пути `mans/i.$s`, где `$s` и `$i` — переменные. Флаг `-f` — это проверка “является ли путь файлом (не папкой)”.
7. Оба выполняют цикл, но условие выхода противоположное:
 - `while` — работает, пока условие верно (истина)